RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

ASSIGNMENT - 4

Prepared by: Mohamed B Sirajuddeen

Reg No : 22TD0053

Subject Name : Embedded Systems

Subject Code : CST62

Year/Sem/Sec : Y3/S6/B

Date of Submission: 13/5/2025

Unit : 4

Teacher's Sign :

Department Vision and Mission

Vision

Cultivate student as technocrats, researchers and entrepreneurs in the field of computer science and engineering.

Missions

- To impart quality education in Computer Science and Engineering by means of learning techniques and value-added courses.
- To inculcate professional excellence and research culture by encouraging projects in cutting-edge technologies through industry interactions.
- To build leadership skills, ethical values and teamwork among the students.
- To strengthen the collaboration of department and industry through internships, mentorships and professional body activities.

1) Write a simple C program using function calls.

The ARM - Thumb procedure call standard (ATPCS) is the one which tells about how to pass function arguments and return values in ARM registers.

The recently introduced ARM thumb procedure call standard covers both the ARM and thumb interworking more efficiently.

The first four integer arguments are placed in the first four ARM registers $r0, r1, r2$ and $r3$.

The subsequent integer arguments are placed on the full descending stack ascending in memory.

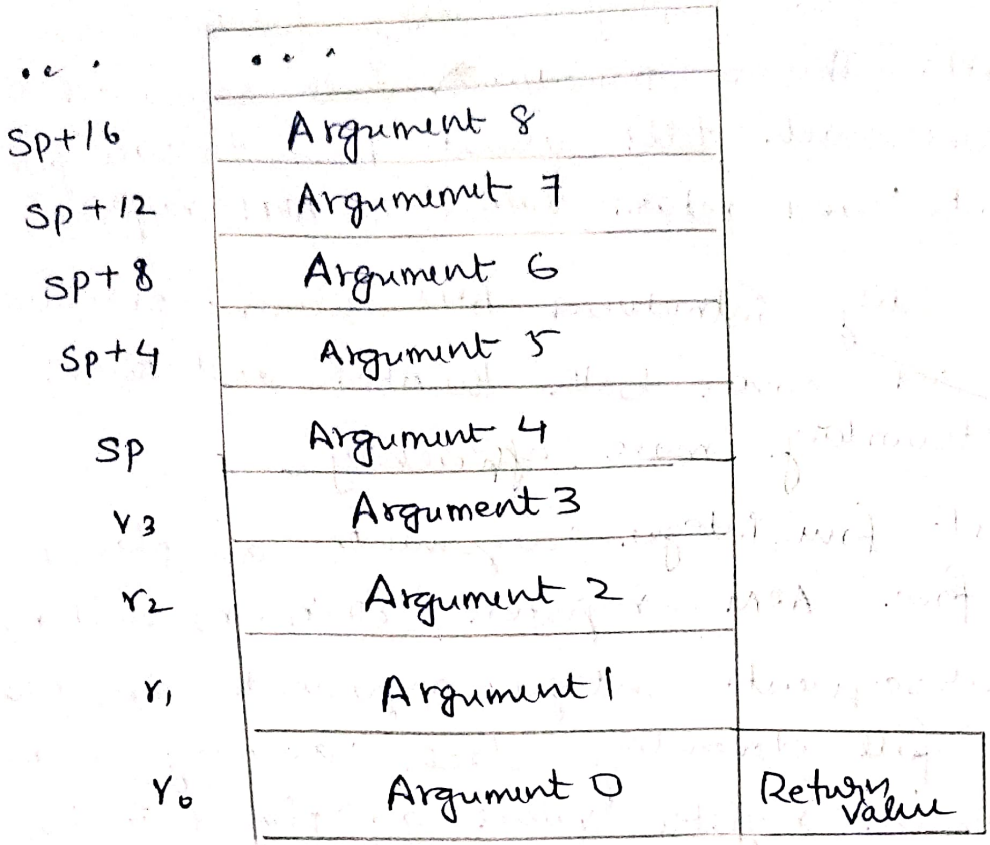
The function register values are placed in $r0$.

For a two word argument like long or double a pair of consecutive argument register are returned in register $r0, r1$.

Now, according to the command line compiler option the compiler may pass structure in registers

In ARM PCS, it is easier to call functions with four or fewer arguments than functions with four or fewer arguments the compiler can pass all the arguments in the register.

If a C function needs more than four arguments, then it is beneficial to use structures in this context, first the group pointer was passed rather than passing the multiple arguments, because the arguments that are passed will depend on the structure of the software.



Example:

The typical routine to insert N bytes from array data into a queue we implement the queue using a cyclic buffer with start address : start (inclusive) and end address & end (exclusive)

```
char * queue_bytes_V1 (
    char * Q_start  /* Queue buffer start
address */ char * Q_end,  /* Queue buffer end
address */
    char * Q_ptr        /* current queue pointer
position */ char * data,  /* data to insert into
the queue */ unsigned int N ) /* Number of
                                 bytes to
insert */
{ do
{   *(Q_ptr++) = *(data++);
    if (Q_ptr == Q_end)
{
```

Q-ptr = Q-start;

}

while (--N);

return Q-ptr;

? This compiles to

```
queue_byte_V1
    STR     r14, [r13, #-4]!    ; save lr on the stack.
    LDR     r12, [r13, #4]      ; r12 = N

queue_V1_loop
    LDRB    r14, [r3], #1       ; r14 = *(data++)
    STRB    r14, [r2], #1       ; *(Q_ptr++) = r14
    CMP     r2, r1              ; if (q_ptr == Q_end)
    MOVEQ   r2, r0              ; {Q_ptr = Q_start;}
    SUBS    r12, r12, #1        ; --N and set flags.
    BNE     queue_V1_loop       ; if (N != 0) goto loop
    MOV     r0, r2              ; r0 = Q_ptr
    LDR     pc, [r13], #4       ; return r0
```

The caller function need not preserve register that it can see the called doesn't corrupt.

Therefore the caller function need not save all the ATPCS corruptible register

If the caller function is very small then the compiler can inline the code in the caller function

This removes the function call overhead completely.

2) ARM programming using Integers and floating point arithmetic.

ARM programming using integers.

The ARM's load and store instructions are only guaranteed to load and store value with address aligned to the size of the access width.

ARM compiler will automatically align the start address of a structure to a multiple of the largest access width used within the structure and align within structures to their access width by inserting padding.

For instance consider the structure

```
struct {
    char a;
    int b;
    char c;
    short d;
};
```

The layout of the memory system is

Address	+3	+2	+1	+0
+0	pad	pad	pad	a
+4	b [31,24]	b [23,16]	b [15,8]	b [7,0]
+8	d [15,8]	d [7,0]	pad	c

To improve the memory usage let us rearrange the elements of structure.

```
struct
{ char a;
  char c;
  short d;
  int b;
}
```

This reduces the structure size from 12 bytes to 8 bytes with the following new layout is

Address	+3	+2	+1	+0
+0	d [15, 3)	d [7,0]	c	a
+4	b [31,24]	b [23,16]	b [15, 3]	b [7,0]

It is good idea of group structure elements of the same size that the structure layout doesn't contain unnecessary padding.

Floating point Arithmetic

The ARM core does not contain any actual floating point hardware. Instead there are three options for an application which needs floating point support

They are:

1) Floating-point accelerator (FPA) Hardware co-processor

This implements a floating-point instruction set using a number of ARM co-processor instructions. However, this does not require the FPA hardware to exist within the system as a co-processor.

2. Floating point Emulator (FPE)

FPE emulates in software the instruction that the FPA executes. This means that there is no need to recompile code for systems with or without the FPE.

3. Floating-point Library (FP lib)

Floating-point operations are compiled into function calls, to library routines than floating-point instructions. Although this is slower that using a FPE. it is typically two or three times faster than using the FPE.

The overall code size of the system is also smaller because only the required library routines are included rather than whole of the FPE.

Therefore the floating point library is the route that ARM recommends for use is embedded system and is the default one.

The recommended compiler option gives the best results in terms of performance and code size.

3) ARM programming using C in pointer and register allocation pointers:

Pointers

- Pointers are powerful part of C language.
- If the address of the variable is taken the compiler must assume that the variable can be change by any assignment through a pointer or by any function call. making it impossible to put into a register.

This is also true for global variables as they might have their address taken in some other function. This problem is known as pointer aliasing because the pointer is known as an alias of the variable it point to.

Eg: void timer (int *t1, int *t2, int *step)

```
{ *t1 += *step;
  *t2 += *step;
}
```

This compiles to,

```
Timer2  LDR  R3 [R0] ; r3 = *t1
        LDR  r12, [r2] ; r12 = *step
        ADD  r3, r3, r12 ; r3 += r12
        STR  r3, [R0] ; *t1 = r3
        LDR  r0 [r1] ; r0 = *t2
        LDR  r2, [r2] ; r2 = *step
        ADD  r0, r2 ; r0 += r2
        STR  r0, [r1] ; *t2 = r0
        MOV  PC, r14 ; return.
```

Here we would expect that *step to be pulled from memory once and used twice. That doesn't happen. Because one is written to cache. step is a local variable, the redundant load is eliminated.

The same problem arises if we use struc accesses instead of direct pointer access. The following code also compiles inefficiently.

```
typedef struct {int step;} state;
typedef struct {int t1,t2;} time;
void timer2 (state * state1, time * time1)
```

```
{ timer -> t1 += state1 -> step;
  timer -> t2 += state -> step }
```

The compiler evaluates state -> step twice in case state -> step and timer -t1 are at the same memory address.

The solution is to create a new local variable to hold the value of state -> state so that the compiler performs only a single load.

Register allocation

→ In order to hold general purpose data we can use 14 out of the 16 visible ARM register. The two registers that were not are r13 and r15.

Allocating variables to Register Number

While writing an assembly routine, it is better to start by using names for the variables rather than explicit register numbers. Registers names increase the clarity and readability of optimized code.

Making the most of available registers

On load-store architecture such as the ARM, it is more efficient to access values held in registers than value held in memory.

There are several tricks we can use to fit several 32 bit length variables into a single 32-bit register and these can reduce code size and increase performance.